

Software Requirements and Design Document

For

Group <27>

Version 1.0

Authors:

Carter Venclauskas
Nate Thomas
Julian Sierra

1. Overview (5 points)

The system is a 2D action-adventure metroidvania game built in C# using the Unity engine. It is designed to emulate the exploration and key/lock style of gameplay found in other metroidvanias like Super Metroid and Hollow Knight. It will emphasize responsive controls and smooth character animations. It will obviously be designed with object oriented principles.

2. Functional Requirements (10 points)

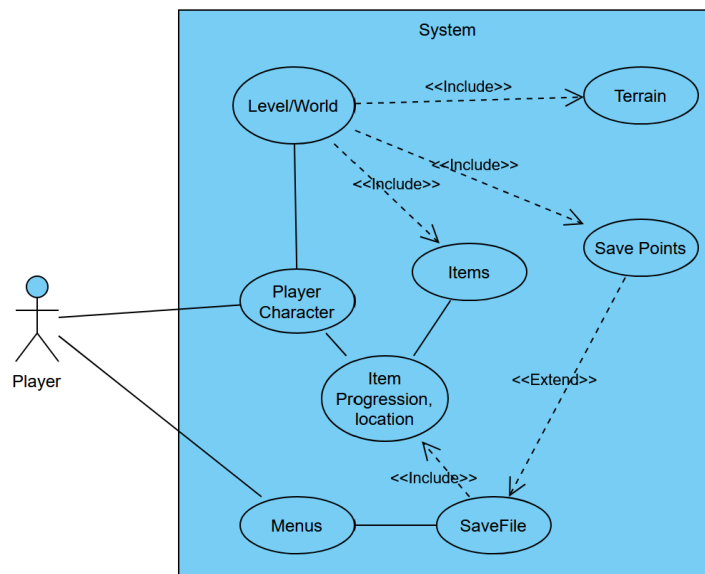
1. It should have responsive character movement (left right jump), including ledge grabbing. - high
2. It should have omnidirectional aiming and shooting mechanics - high
3. It should have player animations based on current movement state and weapon aiming position - high
4. It should have multiple levels - high
5. It should have a main menu - medium
6. It should have a save/load system - low
7. It should have sound effects for weapons movement, and some environment - low

3. Non-functional Requirements (10 points)

1. Game should support new additions without refactoring
2. Game should run at least 60 fps
3. Game should not crash during gameplay
4. HUD elements should be at least 1080p
5. Code should utilize OOP principles for maximum reusability

4. Use Case Diagram (10 points)

The only user is players, where they interact with the system via the main menu and the player character. The main menu allows you to start a new save file or load an existing one. A save file contains information on a player character's item progression and their location in the world. Once a save file is loaded, the player will then interact with the system via the player controller, which acts as the proxy for the player to interact with the level or world and its components. Levels/world contains items that count towards item progression, terrain, and save points which are places to update the loaded save file.



5. Class Diagram and/or Sequence Diagrams (15 points)

This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.

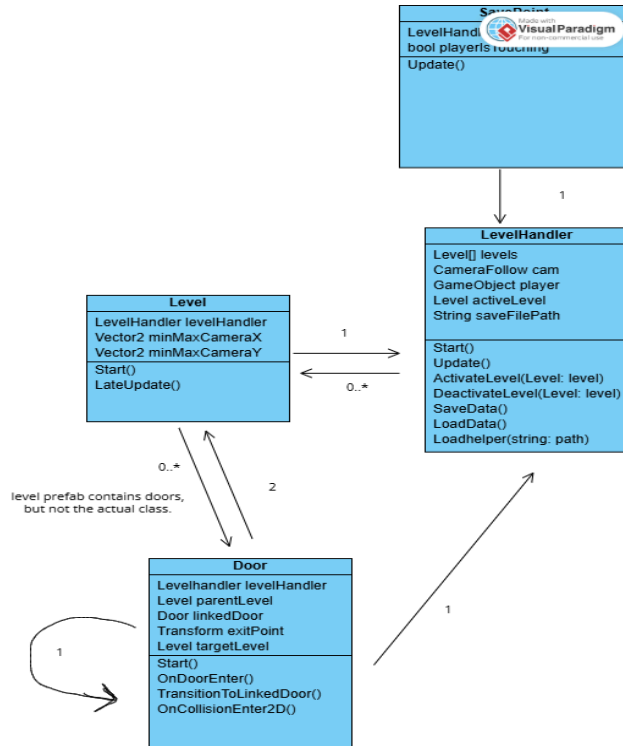
If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram of the entire system** and **Sequence Diagrams for the three (3) most important use cases in your system**.

If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams**, but for **all the use cases of your system**. In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.

Class Diagrams show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.

A **Sequence Diagram** simply depicts **interaction between objects** (or **functions** - in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function.

It's important to keep in mind Unity's GameObject/Hierarchy/Prefab system. While a class may not directly use another class, it may have child objects of that class type. You can see that relationship with Level having 0..* Doors, despite not having Door object variables. Doors are stored in Level "prefabs", and thus are children of Levels, and are enabled and disabled with the level.



UML class diagram showing the basic level and save/load system.

Basic relationships:

Player controller:

1. PlayerMovement
 - a. Handles movement, player physics, calls state machine
 - i. Depends on PlayerStateMachine and ProgressionGlobals
 - ii. Has a NormalState and LedgeGrabState object
2. PlayerState
 - a. Abstract class, has an enter, exit, and update function for state behavior
 - i. Depends on PlayerMovement class
 - b. Subclasses: NormalState, LedgeGrabState
3. PlayerStateMachine
 - a. Switches between player states (normal movement vs. ledge grab)
 - i. Calls back into PlayerMovement to move player

Shooting

4. Projectiles can be fired into 8 directions (45 degree steps).
5. Uses current direction calculated in PlayerMovement
6. AnimationLogic reads from both PlayerMovement and Shooting for choosing between the two separate animators in the legs and torso
7. Projectile
 - a. .cs and prefabs instantiated by Shooting.cs

ProgressionGlobals

8. Static class that holds progression related bools
9. Used by save/load system in LevelHandler.
10. Currently used for gameplay only in PlayerMovement
11. Interacts with ProgressionItems (ProgressionItems change the bools)

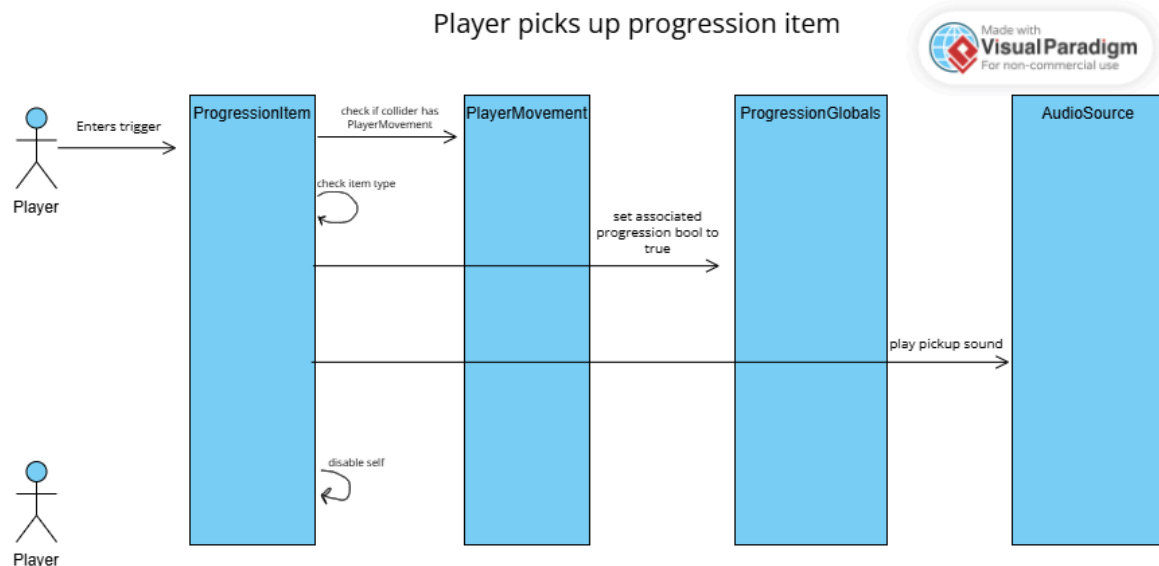
ProgressionItem

1. Association with ProgressionGlobals
2. Dependency on PlayerMovement (checks if player is colliding with it)

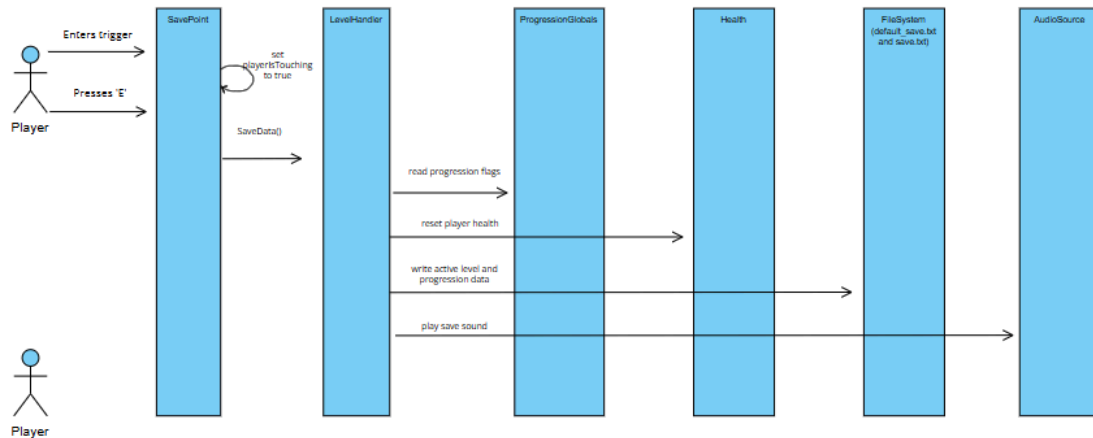
CameraFollow

1. Uses Player Transform (not one of our classes)
2. ClampCameraPosition() used by LevelHandler to clamp the camera position based on Level max camera values

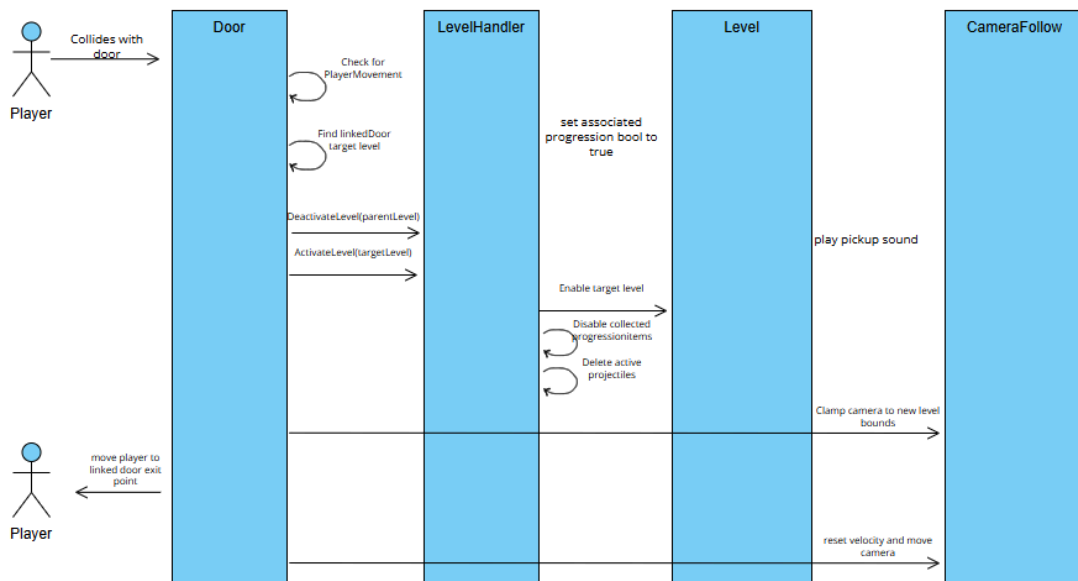
Sequence diagrams:



Player saves progress



Player transitions through a door



6. Operating Environment (5 points)

The software will run on a PC environment using Windows (10 and 11). It is developed in the Unity game engine and requires hardware that is capable of running Unity applications, including basic graphics support. The software should not have any difficulty running alongside other applications for any particular reason.

7. Assumptions and Dependencies (5 points)

Given the fact that our project was based in one software system, using C# and instructions directly correlated with Unity, our team was short of assumed factors that plainly affected our functional requirements. However, In the initial project layout, our scope was too grand, and certain goals were not met.